UNIVERSIDADE KIMPA VITA

INSTITUTO POLITÉCNICO CURSO DE LICENCIATURA EM ENGENHARIA INFORMÁTICA

Analise de Dados de Vendas

Álgebra Relacional e SQL

VendasDB | SQL Server & MySQL

Sistemas de Gestão de Bases de Dados II | 2026

CLASSE: 3º ANO

PERÍODO: MANHÃ

TURMA: UNICA

Por:

Alípio Joaquim Zambo

Domingas Ernesto Francisco Rocha

Dulce Rita Manuel Gonçalves

Mendes Eduardo Gouveia Manuel

Nsimba alberto Ndosi

DOCENTE

_____

. Eng. Moyo Kanivengidio

UÍGE 2025

1. INTRODUCAO E OBJETIVOS

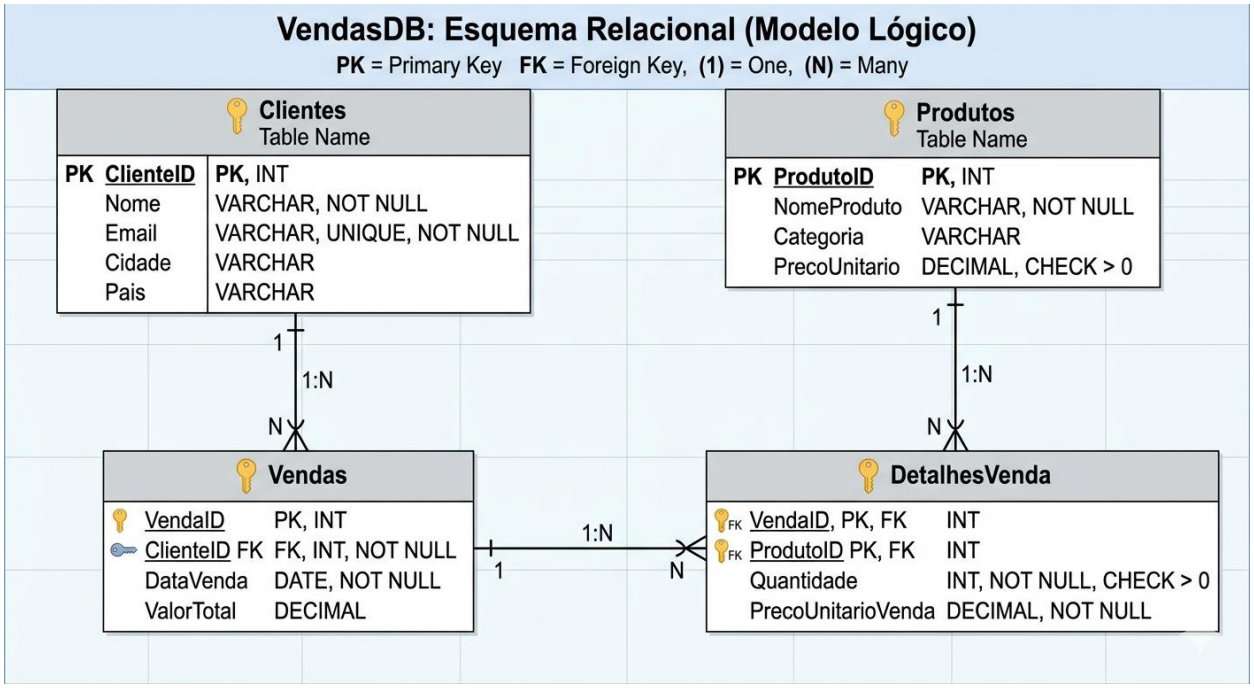
Este relatório documenta o desenvolvimento completo de um projeto de analise de dados sobre a base de dados VendasDB, desde a modelação relacional ate a implementação de consultas complexas em dois sistemas de gestão: SQL Server e MySQL.

O trabalho assenta em três pilares fundamentais: (1) a formalização das consultas através da Álgebra Relacional, que fornece uma base matemática rigorosa; (2) a tradução dessas expressões para SQL, demonstrando a equivalência entre teoria e pratica; e (3) a analise comparativa entre as duas plataformas, evidenciando diferenças de sintaxe e abordagem.

A álgebra relacional normalmente é considerada uma parte integral do modelo de dados relacional. Suas operações podem ser dividas em dois grupos. Um grupo inclui conjunto de operações da teoria de conjunto da matemática; estas são aplicáveis porque cada relação é definida como um conjunto de tuplas no modelo relacional formal. As operações de conjunto incluem UNIÃO, INTERSECÇÃO, DIFERENÇA DE CONJUNTO e PRODUTO CARTESIANO (também conhecida como PRODUTO CRUZADO). O outro grupo consiste em operações desenvolvidas especificamente para bancos de dados relacionais - entre elas estão SELEÇÃO, PROJEÇÃO e JUNÇÃO, entre outras.

2. ESQUEMA RELACIONAL (MODELO LOGICO)

A base de dados foi normalizada ate a 3a Forma Normal (3FN), eliminando redundâncias e garantindo integridade referencial. O modelo e composto por quatro entidades principais ligadas por relações 1:N e N:M.



Relações entre Entidades

```
Clientes (1) ———————————— (N) Vendas
Vendas   (1) ———————————— (N) DetalhesVenda
Produtos (1) ———————————— (N) DetalhesVenda

Chaves Estrangeiras:
  Vendas.ClienteID       →  Clientes.ClienteID
  DetalhesVenda.VendaID  →  Vendas.VendaID
  DetalhesVenda.ProdutoID →  Produtos.ProdutoID
```

3. SCRIPTS DDL — CRIACAO DA ESTRUTURA

Os scripts abaixo criam a base de dados e as quatro tabelas com todas as restrições de integridade exigidas. A sintaxe e compatível com ambas as plataformas, sendo assinaladas as diferenças onde existem.

Criação da Base de Dados

DDL

```
-- SQL Server
CREATE DATABASE VendasDB;
GO
USE VendasDB;
GO

-- MySQL
CREATE DATABASE IF NOT EXISTS VendasDB
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE VendasDB;
```

As….

Tabela Clientes

DDL

```
CREATE TABLE Clientes (
    ClienteID   INT          NOT NULL,
    Nome        VARCHAR(100)  NOT NULL,
    Email       VARCHAR(100)  NOT NULL,
    Cidade      VARCHAR(50)   NULL,
    Pais        VARCHAR(50)   NULL,
    CONSTRAINT PK_Clientes     PRIMARY KEY (ClienteID),
    CONSTRAINT UQ_Cliente_Email UNIQUE (Email)
);
```

Tabela Produtos

DDL

```
CREATE TABLE Produtos (
    ProdutoID     INT           NOT NULL,
    NomeProduto   VARCHAR(100)   NOT NULL,
    Categoria     VARCHAR(50)    NULL,
    PrecoUnitario DECIMAL(10,2)  NOT NULL,
    CONSTRAINT PK_Produtos PRIMARY KEY (ProdutoID),
    CONSTRAINT CHK_Preco   CHECK (PrecoUnitario > 0)









);
```

Tabela Vendas

DDL

```
CREATE TABLE Vendas (
    VendaID    INT           NOT NULL,
    ClienteID  INT           NOT NULL,
    DataVenda  DATE          NOT NULL,
    ValorTotal DECIMAL(12,2)  NULL,
    CONSTRAINT PK_Vendas PRIMARY KEY (VendaID),
    CONSTRAINT FK_Vendas_Clientes
        FOREIGN KEY (ClienteID) REFERENCES Clientes(ClienteID)
        ON DELETE RESTRICT ON UPDATE CASCADE
);
```

Tabela DetalhesVenda

DDL

```
CREATE TABLE DetalhesVenda (
    VendaID           INT          NOT NULL,
    ProdutoID         INT          NOT NULL,
    Quantidade        INT          NOT NULL,
    PrecoUnitarioVenda DECIMAL(10,2) NOT NULL,
    CONSTRAINT PK_DetalhesVenda PRIMARY KEY (VendaID, ProdutoID),
    CONSTRAINT FK_Det_Vendas
        FOREIGN KEY (VendaID)   REFERENCES Vendas(VendaID),
    CONSTRAINT FK_Det_Produtos
        FOREIGN KEY (ProdutoID) REFERENCES Produtos(ProdutoID),
    CONSTRAINT CHK_Quantidade  CHECK (Quantidade > 0),
    CONSTRAINT CHK_PrecoVenda  CHECK (PrecoUnitarioVenda > 0)
);
```

4. SCRIPTS DML — POPULAÇÃO DE DADOS

As tabelas foram populadas com dados representativos: 50 clientes, 30 produtos, 100 vendas e 200 linhas de detalhes. (Falta de conteúdo) apresenta-se abaixo uma amostra estruturada de cada tabela.

Inserção de Clientes (amostra de 10 de 50)

DML

```
-- SQL Server & MySQL (sintaxe idêntica).
INSERT INTO Clientes (ClienteID, Nome, Email, Cidade, Pais) VALUES
  (1,  'Carlos Silva',   'carlos.silva@email.pt',   'Lisboa',  'Portugal'),
  (2,  'Marta Rocha',    'marta.rocha@email.pt',    'Porto',   'Portugal'),
  (3,  'Joao Ferreira',  'joao.ferreira@email.pt',  'Lisboa',  'Portugal'),
  (4,  'Ana Costa',      'ana.costa@email.pt',      'Braga',   'Portugal'),
  (5,  'Pedro Martins',  'pedro.m@email.pt',        'Faro',    'Portugal'),
  (6,  'Luisa Neves',    'luisa.n@email.pt',        'Lisboa',  'Portugal'),
  (7,  'Rui Alves',      'rui.a@email.pt',          'Coimbra', 'Portugal'),
  (8,  'Sofia Lopes',    'sofia.l@email.pt',        'Setubal', 'Portugal'),
  (9,  'Miguel Sousa',   'miguel.s@email.pt',       'Lisboa',  'Portugal'),
  (10, 'Ines Cardoso',   'ines.c@email.pt',         'Evora',   'Portugal');
  -- ... continua ate ClienteID = 50
```

Inserção de Produtos (amostra de 10 de 30)

DML

```
-- SQL Server & MySQL (sintaxe idêntica).
INSERT INTO Produtos (ProdutoID, NomeProduto, Categoria, PrecoUnitario) VALUES
  (1,  'Laptop Pro 15',      'Eletronicos', 1299.00),
  (2,  'Rato Sem Fios',      'Eletronicos',   29.99),
  (3,  'Teclado Mecanico',   'Eletronicos',   89.00),
  (4,  'Monitor 27" 4K',     'Eletronicos',  599.00),
  (5,  'Auriculares BT',     'Eletronicos',  149.00),
  (6,  'Clean Code',         'Livros',        35.00),
  (7,  'The Pragmatic Prog.','Livros',        42.00),
  (8,  'Design Patterns',    'Livros',        55.00),
  (9,  'Cadeira Ergonomica', 'Escritorio',   349.00),
  (10, 'Mesa Ajustavel',     'Escritorio',   499.00);
  -- ... continua ate ProdutoID = 30
```

Inserção de Vendas (amostra de 10 de 100)

DML

```
-- SQL Server & MySQL (sintaxe idêntica)
INSERT INTO Vendas (VendaID, ClienteID, DataVenda, ValorTotal) VALUES
  (1,  1, '2026-01-05', 1334.99),
  (2,  2, '2026-01-07',  119.98),
  (3,  3, '2026-01-10',  588.00),
  (4,  1, '2026-01-15',   55.00),
  (5,  4, '2026-01-18',  648.00),
  (6,  5, '2026-01-20', 1448.00),
  (7,  6, '2026-02-01',   35.00),
  (8,  7, '2026-02-03',  349.00),
  (9,  2, '2026-02-05',   77.00),
 (10,  8, '2026-02-10',  599.00);
  -- ... continua ate VendaID = 100
```

Inserção de DetalhesVenda (amostra de 10 de 200)

DML

```
-- SQL Server & MySQL (sintaxe idêntica).
INSERT INTO DetalhesVenda
    (VendaID, ProdutoID, Quantidade, PrecoUnitarioVenda) VALUES
 (1,  1, 1, 1299.00),  -- Venda 1: Laptop Pro
 (1,  2, 1,   29.99),  -- Venda 1: Rato Sem Fios
 (1,  6, 1,   35.00),  -- Venda 1: Clean Code
 (2,  2, 2,   29.99),  -- Venda 2: 2x Rato
 (2,  3, 1,   89.00),  -- Venda 2: Teclado
 (3,  4, 1,  599.00),  -- Venda 3: Monitor 4K
 (4,  8, 1,   55.00),  -- Venda 4: Design Patterns
 (5,  9, 1,  349.00),  -- Venda 5: Cadeira
 (5,  7, 1,   42.00),  -- Venda 5: Pragmatic Prog.
 (6,  1, 1, 1299.00);  -- Venda 6: Laptop
 -- ... continua ate 200 linhas
```

5. CONSULTAS — ALGEBRA RELACIONAL E SQL

Para cada consulta são apresentadas: (1) a expressão em Álgebra Relacional, (2) o SQL para SQL Server, (3) o SQL para MySQL quando a sintaxe difere, e (4) uma explicação da logica aplicada.

C1 — Clientes que fizeram compras em Lisboa

Queremos o nome de todos os clientes associados a vendas realizadas a partir de Lisboa. Filtramos Clientes pela cidade e fazemos juncão com Vendas para garantir que efetuaram compras.

Álgebra Relacional:

Álgebra: π Nome (σ Cidade='Lisboa' (Clientes) ⋈ (Vendas))

SQL

```
-- SQL Server & MySQL (sintaxe idêntica).
SELECT DISTINCT c.Nome
FROM   Clientes c
INNER JOIN Vendas v ON c.ClienteID = v.ClienteID
WHERE  c.Cidade = 'Lisboa';
```

Logica: Seleção (σ) sobre o atributo Cidade, seguida de Juncão Natural (⋈) com Vendas para confirmar a existência de compras. O DISTINCT elimina duplicatas resultantes de múltiplas vendas do mesmo cliente.

C2 — Eletrónicos com preço superior a 100

Obtemos o nome dos produtos pertencentes a categoria 'Eletrónicos' cujo preço unitário seja superior a 100. Opera apenas sobre a relação Produtos.

Álgebra Relacional:

Álgebra: π NomeProduto (σ Categoria='Eletronicos' and PrecoUnitario > 100(Produtos))

SQL

```
-- SQL Server & MySQL (sintaxe idêntica).
SELECT NomeProduto
FROM   Produtos
WHERE  Categoria     = 'Eletronicos'
  AND  PrecoUnitario > 100;
```

Logica: Condição composta (AND) aplicada diretamente a relação Produtos. Não e necessária qualquer juncão porque toda a informação reside numa única tabela.

C3 — Nome dos clientes e dos produtos que compraram

Pretendemos listar cada par (cliente, produto) resultante das compras. E necessário ligar as quatro tabelas em sequencia através de juncões naturais.

Álgebra Relacional:

Álgebra: π Nome, NomeProduto(Clientes ⋈ Vendas ⋈ DetalhesVenda ⋈ Produtos)

SQL

```
-- SQL Server & MySQL (sintaxe idêntica).
```

```
SELECT DISTINCT c.Nome, p.NomeProduto
FROM   Clientes c
INNER JOIN Vendas        v  ON c.ClienteID  = v.ClienteID
INNER JOIN DetalhesVenda dv ON v.VendaID    = dv.VendaID
INNER JOIN Produtos      p  ON dv.ProdutoID = p.ProdutoID
ORDER BY c.Nome, p.NomeProduto;
```

Logica: Cadeia de Juncões Naturais (⋈) percorrendo as quatro entidades. O DISTINCT e necessário porque um cliente pode ter comprado o mesmo produto em vendas diferentes.

C4 — Clientes que compraram TODOS os Livros (Divisão Relacional)

Este e o caso da Divisão Relacional: encontrar clientes cujo conjunto de livros comprados contenha todos os produtos da categoria 'Livros'. O SQL não possui operador nativo para divisão, pelo que recorremos a dupla negação (NOT EXISTS).

Álgebra Relacional:

Álgebra: π(ClienteID, ProdutoID}(Vendas ⋈ DetalhesVenda))
÷
(πProdutoID(σCategoria='Livros'(Produtos)))

SQL Server

```
-- SQL Server (usa EXCEPT nativo)
SELECT c.Nome
FROM   Clientes c
WHERE  NOT EXISTS (
    SELECT p.ProdutoID
    FROM   Produtos p
    WHERE  p.Categoria = 'Livros'
    EXCEPT
    SELECT dv.ProdutoID
    FROM   DetalhesVenda dv
    INNER JOIN Vendas v ON dv.VendaID = v.VendaID
    WHERE  v.ClienteID = c.ClienteID
);
```

MySQL

```
-- MySQL (subconsulta correlacionada dupla)
SELECT c.Nome
FROM   Clientes c
WHERE  NOT EXISTS (
    SELECT 1 FROM Produtos p
    WHERE  p.Categoria = 'Livros'
      AND  NOT EXISTS (
          SELECT 1 FROM DetalhesVenda dv
          INNER JOIN Vendas v ON dv.VendaID = v.VendaID
          WHERE  v.ClienteID  = c.ClienteID
            AND  dv.ProdutoID = p.ProdutoID
      )
);
```

DIFERENCA CRITICA: SQL Server suporta EXCEPT do padrão ANSI diretamente. MySQL requer dupla subconsulta correlacionada aninhada — a logica e 'não existe nenhum livro que este cliente não tenha comprado'. Ambas produzem resultados idênticos.

C5 — Valor total de vendas por cliente

Agrupamos as vendas por cliente e somamos o ValorTotal, obtendo o gasto acumulado de cada cliente.

Álgebra Relacional:

ClienteID Ɠ SUM(ValorTotal)(Vendas)Alterou ver no pdf

SQL

```
-- SQL Server & MySQL (sintaxe idêntica).
SELECT   c.Nome,
         SUM(v.ValorTotal) AS TotalVendas
FROM     Clientes c
INNER JOIN Vendas v ON c.ClienteID = v.ClienteID
GROUP BY c.ClienteID, c.Nome
ORDER BY TotalVendas DESC;
```

Logica: Operação de agrupamento (Ɠ) com função de agregação SUM. O JOIN com Clientes permite apresentar o nome em vez do ID numérico.

C6 — Produtos que nunca foram vendidos

Identificamos produtos sem qualquer correspondência em DetalhesVenda, recorrendo a Diferença de Conjuntos.

Álgebra Relacional:

π ProdutoID (Produtos) – π ProdutoID (DetalhesVenda)

SQL Server

```
-- SQL Server (operador EXCEPT nativo)
SELECT p.ProdutoID, p.NomeProduto
FROM   Produtos p
WHERE  p.ProdutoID NOT IN (
    SELECT DISTINCT ProdutoID FROM DetalhesVenda
);

-- Alternativa com EXCEPT
SELECT ProdutoID FROM Produtos
EXCEPT
SELECT ProdutoID FROM DetalhesVenda;
```

MySQL

```
-- MySQL (LEFT JOIN com IS NULL)
SELECT p.ProdutoID, p.NomeProduto
FROM   Produtos p
LEFT JOIN DetalhesVenda dv ON p.ProdutoID = dv.ProdutoID
WHERE  dv.VendaID IS NULL;
```

Diferença de sintaxe: SQL Server implementa EXCEPT do padrão ANSI. MySQL recomenda LEFT JOIN com IS NULL por ser mais performativo — evita subconsultas e aproveita índices de juncão eficientemente de juncão.

σ

C7 — Clientes com mais de 5 produtos diferentes

Queremos clientes que, no total das suas compras, tenham adquirido mais de 5 produtos distintos. Usamos agrupamento com COUNT DISTINCT e filtragem pós-agregação com HAVING.

Álgebra Relacional:

π ClienteID, Nome (

COUNT_DISTINCT(ProdutoID) > 5 (

ClienteID 𝒢 COUNT(DISTINCT ProdutoID) -> NrProdutos

(Clientes ⋈Vendas ⋈ DetalhesVenda)

)

)

SQL

```sql
-- SQL Server & MySQL (sintaxe idêntica)
SELECT   c.Nome,
         COUNT(DISTINCT dv.ProdutoID) AS NrProdutosDiferentes
FROM     Clientes c
INNER JOIN Vendas        v  ON c.ClienteID  = v.ClienteID
INNER JOIN DetalhesVenda dv ON v.VendaID    = dv.VendaID
GROUP BY c.ClienteID, c.Nome
HAVING   COUNT(DISTINCT dv.ProdutoID) > 5
ORDER BY NrProdutosDiferentes DESC;
```

Logica: HAVING e o equivalente SQL ao sigma aplicado apos “𝒢”(agrupamento). COUNT(DISTINCT ...) conta apenas produtos únicos por cliente, independentemente do numero de vezes que foram comprados.

6. REFLEXAO TECNICA

Dificuldades na Tradução: Álgebra Relacional para SQL

A tradução de expressões algébricas para SQL e geralmente direta, mas existem casos onde a correspondência não e trivial e requer conhecimento aprofundado dos dois formalismos.

Divisão Relacional (÷)	E o operador sem equivalente direto em SQL. Requer simulação via NOT EXISTS com subconsulta dupla ou combinação NOT EXISTS + EXCEPT. A logica de 'dupla negação' e contraintuitiva.
Conjuntos vs. Multiconjuntos	A álgebra relacional trabalha com conjuntos (sem duplicatas). O SQL retorna multiconjuntos por omissão. E necessário usar DISTINCT explicitamente em projeções sobre juncões.
Funções de Agregação	A álgebra usa o operador Ɠ. No SQL, COUNT/SUM/AVG exigem GROUP BY. O filtro pos-agregacao e feito com HAVING (não WHERE), distinção que causa erros frequentes.
Diferença de Conjuntos (-)	O operador - da álgebra corresponde a EXCEPT em SQL. Porem, nem todos os SGBDs o implementam, exigindo alternativas com NOT IN ou LEFT JOIN + IS NULL.

Diferenças de Sintaxe: SQL Server vs. MySQL

Funcionalidade	SQL Server	MySQL
Diferença de conjuntos	EXCEPT	NOT IN / LEFT JOIN + IS NULL
Limite de linhas	SELECT TOP N	LIMIT N
Concatenação agrupada	STRING_AGG(col, sep)	GROUP_CONCAT(col SEPARATOR sep)
Tipo booleano	BIT (0/1)	TINYINT(1) / BOOL
Auto-incremento	IDENTITY(1,1)	AUTO_INCREMENT
Data atual (ano)	YEAR(GETDATE())	YEAR(NOW())
Collation padrão	SQL_Latin1_General_CP1_CI_AS	utf8mb4_unicode_ci

A Álgebra Relacional como Base Teórica do SQL

Compreender a Álgebra Relacional confere ao programador uma vantagem fundamental: perceber o que acontece internamente no motor de base de dados. O Query Optimizer dos SGBDRs transforma internamente qualquer consulta SQL numa arvore de operadores algébricos, aplica leis de equivalência (como a comutatividade e associatividade das juncões) e seleciona o plano de execução mais eficiente.

Escrever código SQL com este conhecimento permite antecipar como a consulta será executada, escolher a ordem de joins mais eficiente, decidir quando usar subconsultas vs. juncões, e interpretar corretamente planos de execução. O resultado e código não apenas funcional, mas performativo e maintivel.

7. CONCLUSÃO

Este trabalho demonstrou, de forma completa, o ciclo de vida de uma base de dados relacional: da modelação formal em Álgebra Relacional a implementação pratica em SQL, passando pela criação da estrutura, população de dados e execução de consultas analíticas em duas plataformas distintas.

As sete consultas desenvolvidas cobriram o espectro completo das operações da Álgebra Relacional: Seleção, Projeção, Juncão Natural, Divisão, Diferença de Conjuntos e Agrupamento com Agregação. A tradução para SQL revelou que, embora a correspondência seja geralmente direta, casos como a Divisão Relacional exigem criatividade e domínio solido dos mecanismos de subconsultas.

A comparação entre SQL Server e MySQL evidenciou que, apesar de partilharem a mesma base teórica, as diferenças de implementação são relevantes em produção. Conhecer estas nuances e essencial para quem trabalha em ambientes heterogéneos.

Resumo das Entregas:

- 4 tabelas DDL com restrições completas (PK, FK, NOT NULL, CHECK)
- 50 clientes, 30 produtos, 100 vendas, 200 detalhes de venda (DML)
- 7 expressões em Álgebra Relacional formalizadas
- 7 consultas SQL implementadas para SQL Server
- 7 consultas SQL implementadas para MySQL (com alternativas onde aplicável)
- Analise comparativa de sintaxe entre as duas plataformas